Ex 4: Comparative Analysis of Large Language Models

Learning Objective:

Develop a Python program that performs a comparative analysis of Large Language Models by analyzing the text-generation performance of OpenAI, Gemini, and Hugging Face models.

Steps:

1. Import Required Libraries (OpenAI, HuggingFace Transformers, Gemini API)
2. Configure OpenAI key and Google Gemini API key. (This key should be personal. Do not share with anyone).
3. Initialize the Models (OpenAI, Gemini, and Hugging Face).
4. Write function to send prompt and receive response from OpenAI, Gemini, and Hugging Face models
5. Provide a common text prompt. (e.g., “Write a haiku about AI and nature”).
6. Generate Outputs by sending the same prompt to all three models and collect their responses.
7. Display the Model Responses individually. Use Markdown / HTML formatting to display results clearly.
8. Construct an evaluator. (For evaluator you can use Gemini) and produce ranks.
9. Display the ranked model names in best → worst order.

Program:

pip install openai google-generativeai transformers torch

Set API Keys (IMPORTANT)

```
import os

# Set your API keys (DO NOT SHARE THESE)
os.environ["OPENAI_API_KEY"] = "your-openai-key"
os.environ["GOOGLE_API_KEY"] = "your-gemini-key"
```

```
import os
import openai
import google.generativeai as genai
from transformers import pipeline

# ------------------------------
# 1. Configure APIs
# ------------------------------

openai.api_key = os.getenv("OPENAI_API_KEY")
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

# ------------------------------
# 2. Initialize Models
# ------------------------------

# OpenAI (GPT)
def get_openai_response(prompt):
response = openai.ChatCompletion.create(
model="gpt-4o-mini",
messages=[{"role": "user", "content": prompt}],
temperature=0.7
)
return response['choices'][0]['message']['content']


# Gemini
def get_gemini_response(prompt):
model = genai.GenerativeModel("gemini-pro")
response = model.generate_content(prompt)
return response.text


# Hugging Face (local model)
```

```
hf_generator = pipeline("text-generation", model="gpt2")

def get_huggingface_response(prompt):
response = hf_generator(prompt, max_length=50, num_return_sequences=1)
return response[0]['generated_text']


# ------------------------------
# 3. Common Prompt
# ------------------------------

prompt = "Write a haiku about AI and nature"

# ------------------------------
# 4. Generate Outputs
# ------------------------------

print("Generating responses...\n")

openai_output = get_openai_response(prompt)
gemini_output = get_gemini_response(prompt)
hf_output = get_huggingface_response(prompt)

# ------------------------------
# 5. Display Outputs (Markdown-style)
# ------------------------------

print("\n--- Model Responses ---\n")

print("### OpenAI Response:\n")
print(openai_output)

print("\n### Gemini Response:\n")
print(gemini_output)

print("\n### Hugging Face Response:\n")
print(hf_output)

# ------------------------------
# 6. Evaluator using Gemini
# ------------------------------

evaluation_prompt = f"""
You are an evaluator. Rank the following model outputs based on relevance,
creativity, and adherence to the haiku format.

Prompt: {prompt}
```

OpenAI Output:
{openai_output}

Gemini Output:
{gemini_output}

HuggingFace Output:
{hf_output}

Provide ranking in the format:
1. ModelName - Reason
2. ModelName - Reason
3. ModelName - Reason
"""

```
def evaluate_with_gemini(prompt):
model = genai.GenerativeModel("gemini-pro")
response = model.generate_content(prompt)
return response.text

ranking = evaluate_with_gemini(evaluation_prompt)

# --------------------------------
# 7. Display Ranking
# -------------------------------

print("\n--- Model Ranking (Best → Worst) ---\n")
print(ranking)
```

Output:

```
  loader.exec_module(module)
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downlo
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable
config.json: 100% 665/665 [00:00<00:00, 47.8kB/s]
model.safetensors: 100% 548M/548M [00:12<00:00, 50.1MB/s]
Loading weights: 100% 148/148 [00:00<00:00, 420.83it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100% 124/124 [00:00<00:00, 6.69kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 1.14kB/s]
vocab.json: 1.04M/? [00:00<00:00, 16.7MB/s]
merges.txt: 456k/? [00:00<00:00, 10.1MB/s]
tokenizer.json: 1.36M/? [00:00<00:00, 20.0MB/s]
Generating responses...
```

Learning Outcome:

Students will be able to integrate and interact with OpenAI, Gemini, and Hugging Face APIs and evaluate model outputs based on relevance.